

// Fuente: Tema 16. Programación

Arquitectura de Colecciones en Java

Estructuras de datos dinámicas: Cuándo y cómo utilizarlas.

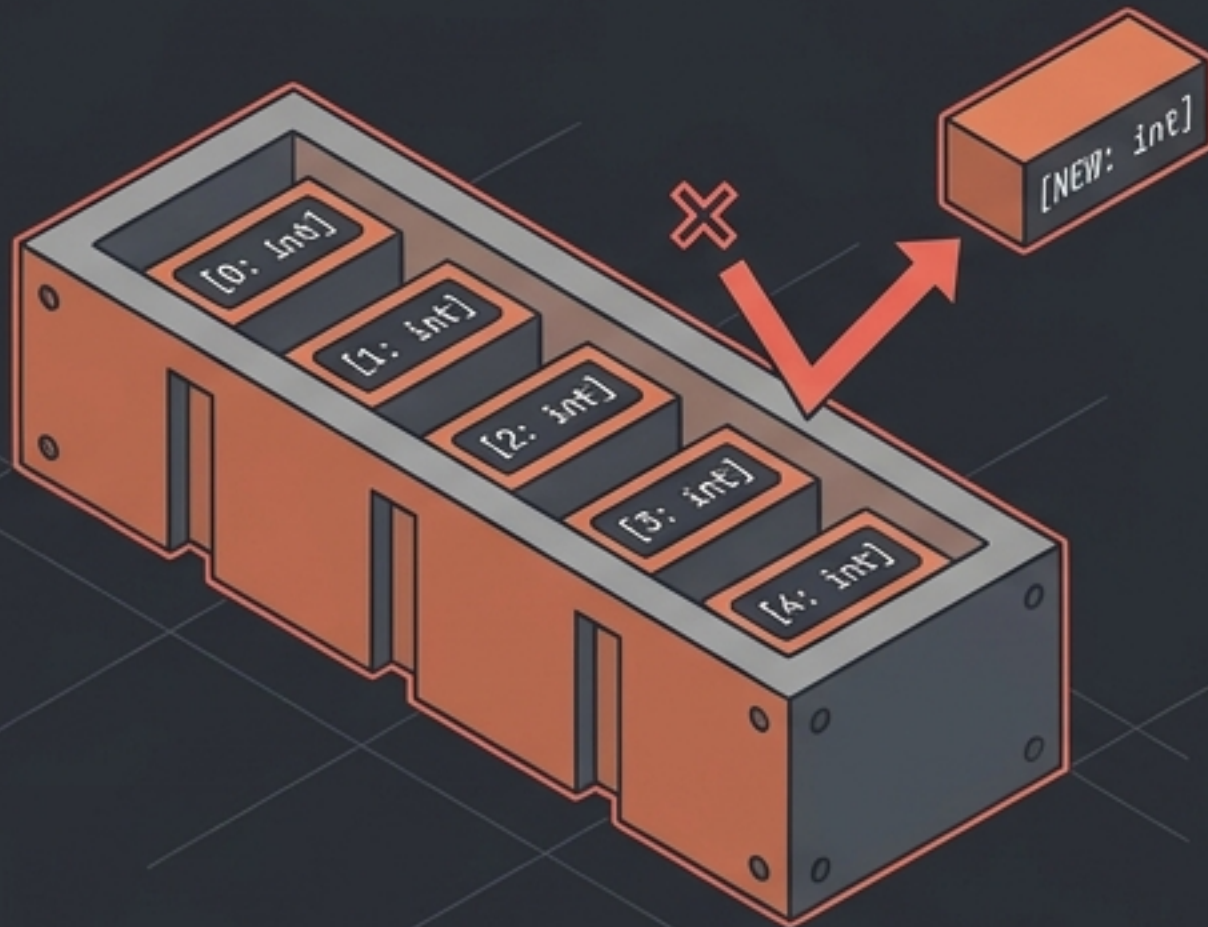
[Target: Java]

[Focus: Data Structures]

[Output: Actionable Framework]

Array Clásico

<CLASSIC_ARRAY_STRUCTURE>



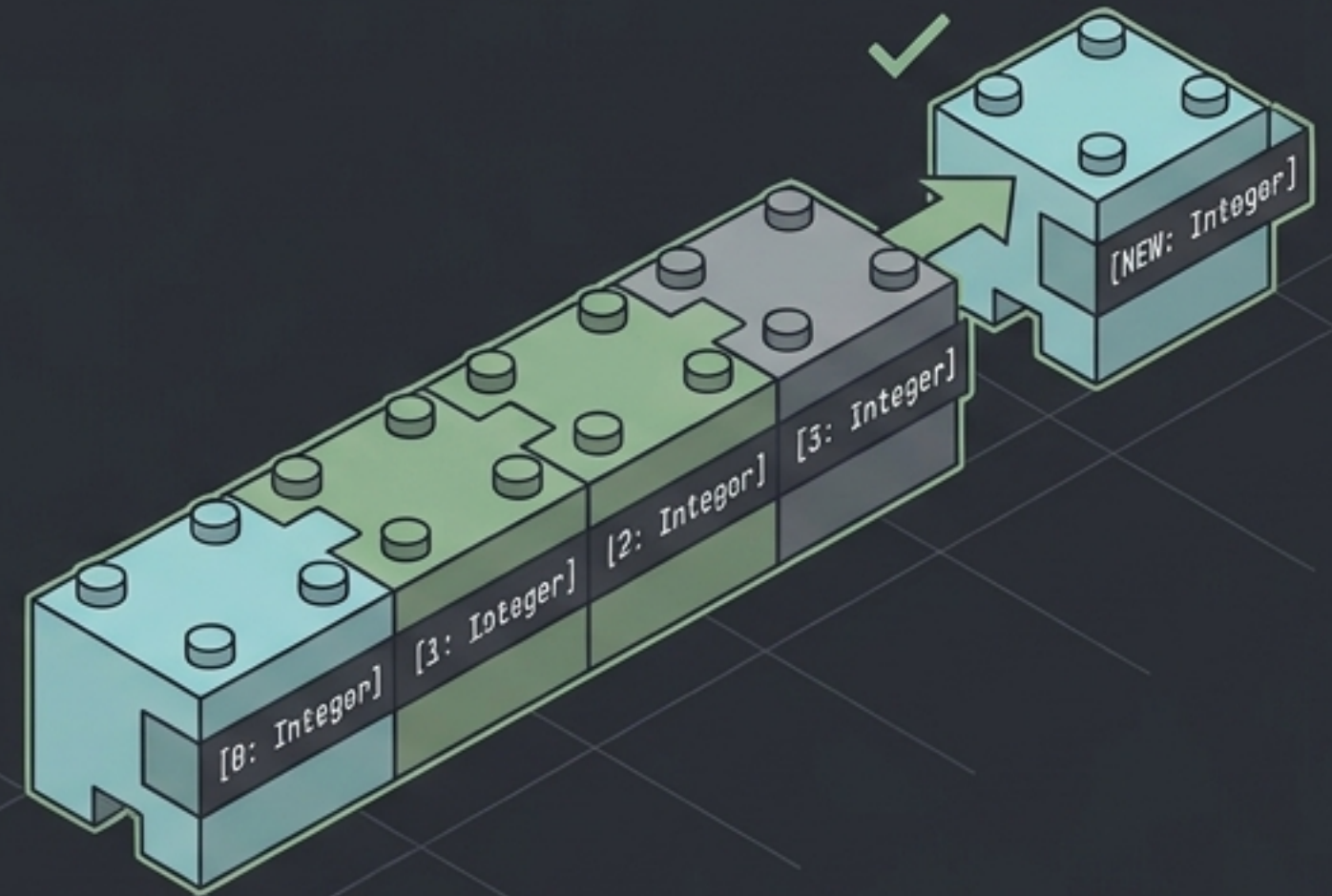
[Tamaño fijo (No puede cambiar una vez creado)]

[Tipos Primitivos (Admite int, double, etc.)]

[Memoria (Almacenamiento contiguo y eficiente)]

Colección (ej. ArrayList)

<COLLECTION_ARRAYLIST_STRUCTURE>



[Tamaño dinámico (Se redimensiona automáticamente)]

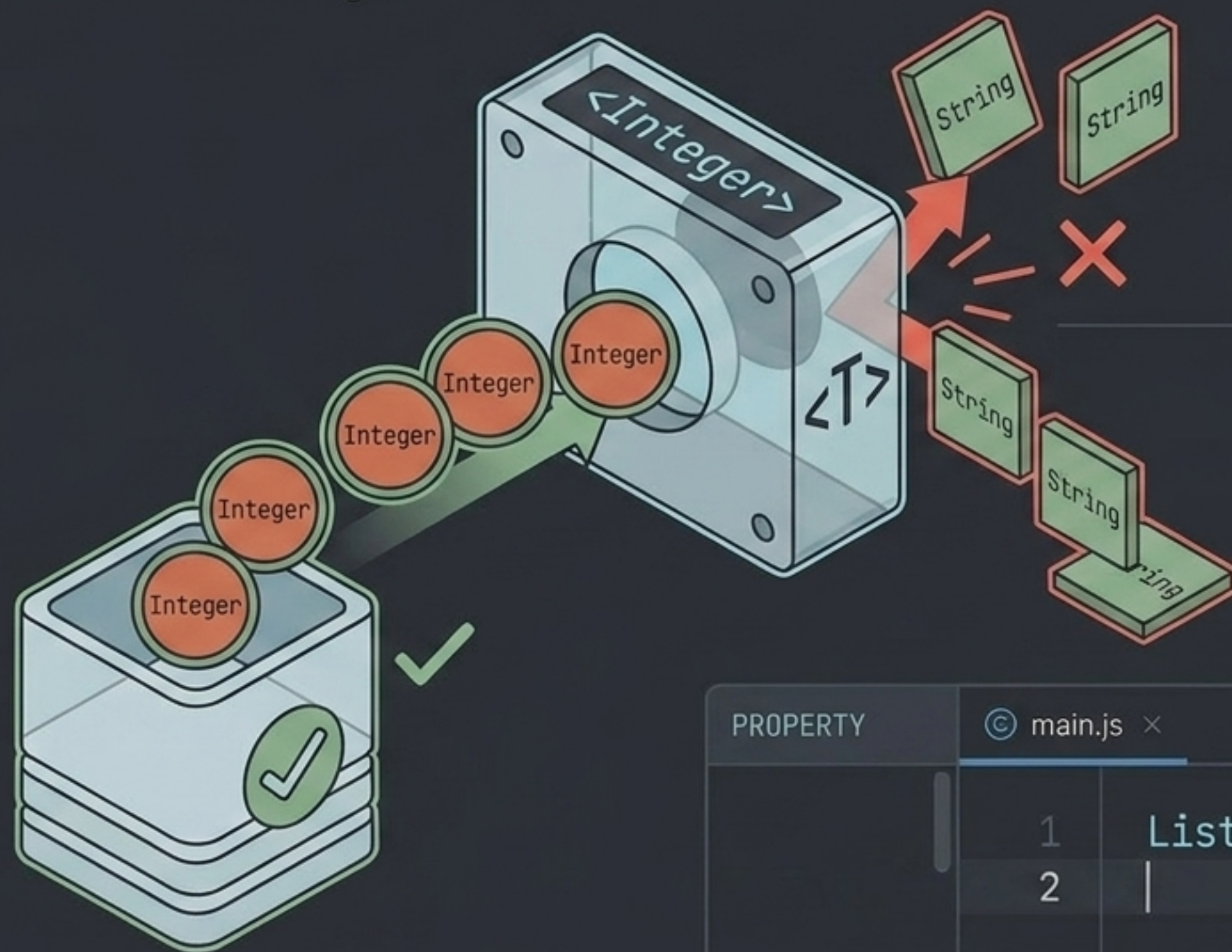
[Solo Objetos (Requiere clases envolventes como Integer)]

[Flexibilidad (Métodos integrados para ordenar, buscar y manipular)]

| <COMPARATIVE_VIEW_DIVIDER> |

Tipos Genéricos $\langle T \rangle$: El Molde de Seguridad

Las colecciones utilizan tipos parametrizados para operar sin necesidad de castings manuales.



PROPERTY

[METADATA]

! RULE

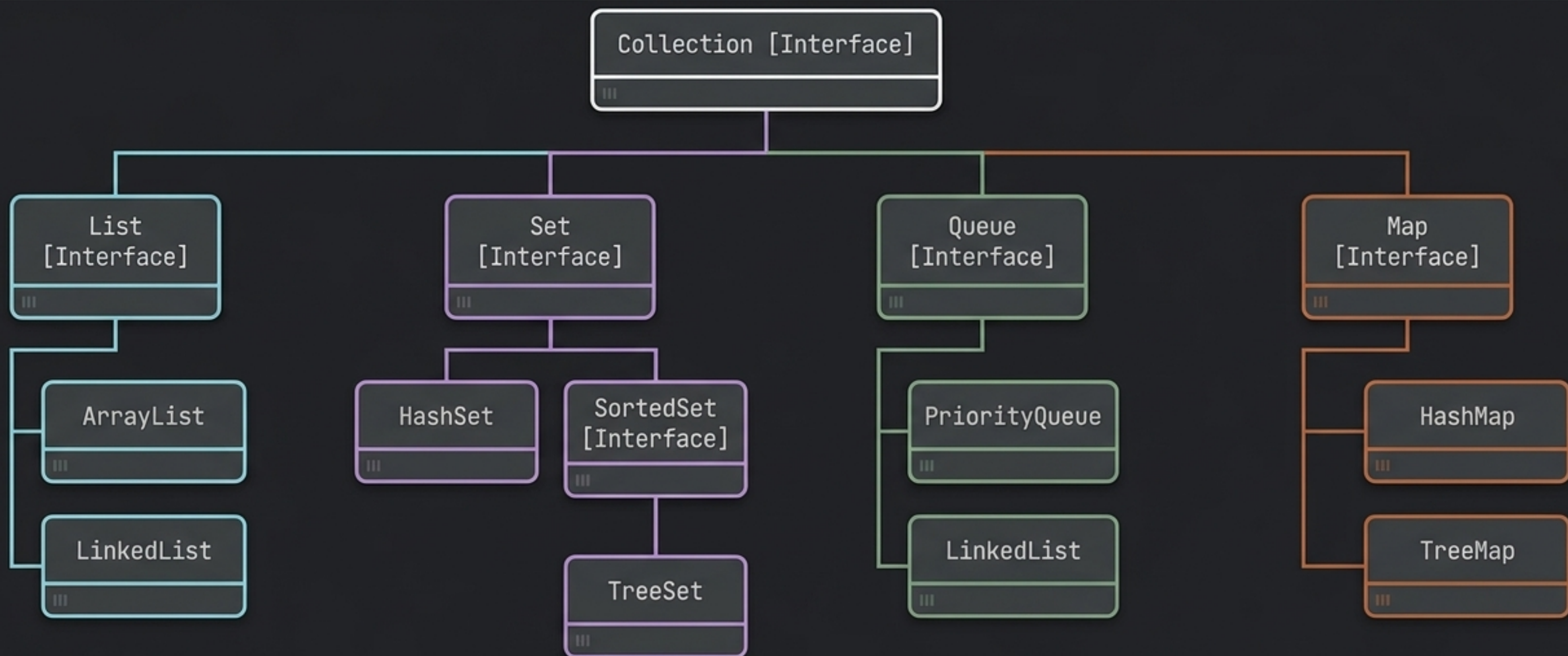
No admiten tipos primitivos directos.
Requieren Wrapper Classes (Clases envoltentes):

```
[ int → Integer ],  
[ double → Double ],  
[ char → Character ].
```

PROPERTY

© main.js ×

```
1 List<Integer> numeros = new ArrayList<>();  
2 |
```

- **List:** Ordenadas, indexadas, admiten duplicados.
- **Set:** Elementos únicos, sin orden garantizado.
- **Queue:** Comportamiento FIFO (First-In, First-Out).
- **Map:** Pares Clave-Valor.

Interfaz List: Secuencias Ordenadas e Indexadas

[Duplicados: SÍ]

[Orden: Inserción]

[Acceso: Por Índice]



Secuencia Indexada

`add(E)`

Agrega al final.

`remove(Object)`

Elimina la primera
ocurrencia.

`get(int)`

Recupera por posición.

`indexOf(Object)`

Busca la posición.

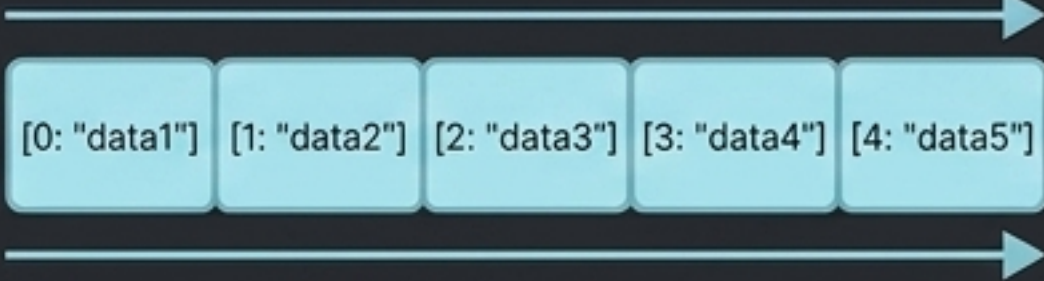
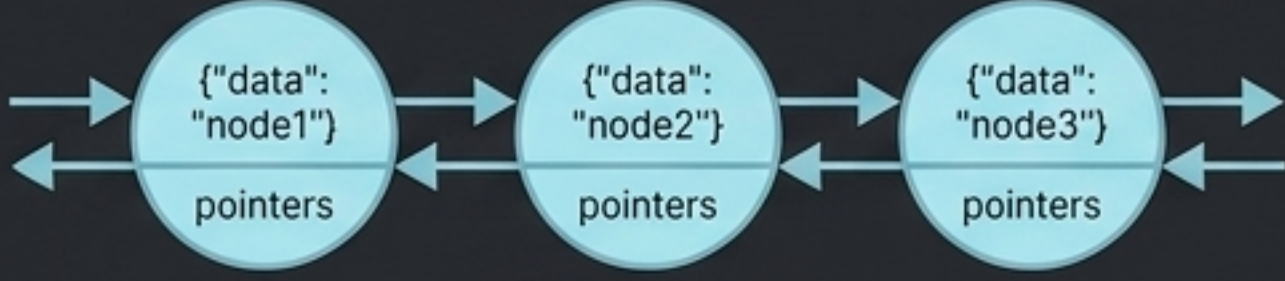
`set(int, E)`

Reemplaza elemento.

`subList(from, to)`

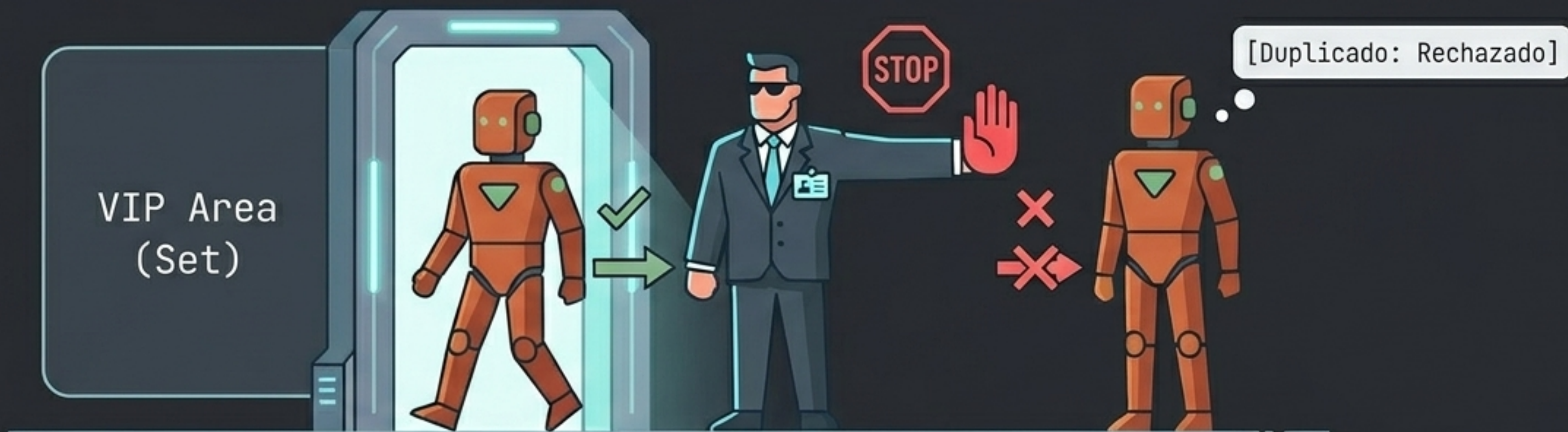
Extrae una porción.

Comparativa: ArrayList vs. LinkedList

	ArrayList	LinkedList
	 <code>// Contiguous memory block</code>	 <code>// Doubly linked nodes</code>
Estructura Interna	Array Dinámico [Array-based]	Lista Doblemente Enlazada [Node-based]
Velocidad de Acceso (Lectura)	Muy Rápido (Acceso aleatorio por índice) [O(1) Access]	Lento (Acceso secuencial nodo a nodo) [O(n) Access]
Inserción / Eliminación (En medio)	Costoso (Requiere desplazar elementos) [O(n) Shift]	Eficiente (Solo cambia los punteros de los nodos) [O(1) Pointer Change]
Consumo de Memoria	Menor [Lower Overhead]	Mayor (Sobrecarga por almacenar referencias a nodos vecinos) [Higher Overhead] due to pointer

Interfaz Set: La Regla de la Unicidad

Un Set rechaza automáticamente cualquier intento de añadir un elemento que ya existe en la colección.



[Duplicados: NO]

[Orden: Depende de la implementación]

Operaciones: Ideal para matemáticas de conjuntos (unión, intersección, diferencia).

Implementaciones de Set

HashSet



Estructura: Tabla Hash.

Orden: Ninguno (Impredecible).

Rendimiento: $O(1)$ Muy rápido
(Búsquedas, inserciones).

TreeSet



Estructura: Árbol Binario
Balanceado.

Orden: Natural (Alfabético/
Numérico o vía Comparable).

Rendimiento: Búsqueda
eficiente, pero más lento que
Hash.

LinkedHashSet



Estructura: Hash + LinkedList.

Orden: Orden de Inserción.

Rendimiento: Similar a HashSet,
con ligera sobrecarga de
memoria.

Interfaz Map: Diccionarios de Clave-Valor

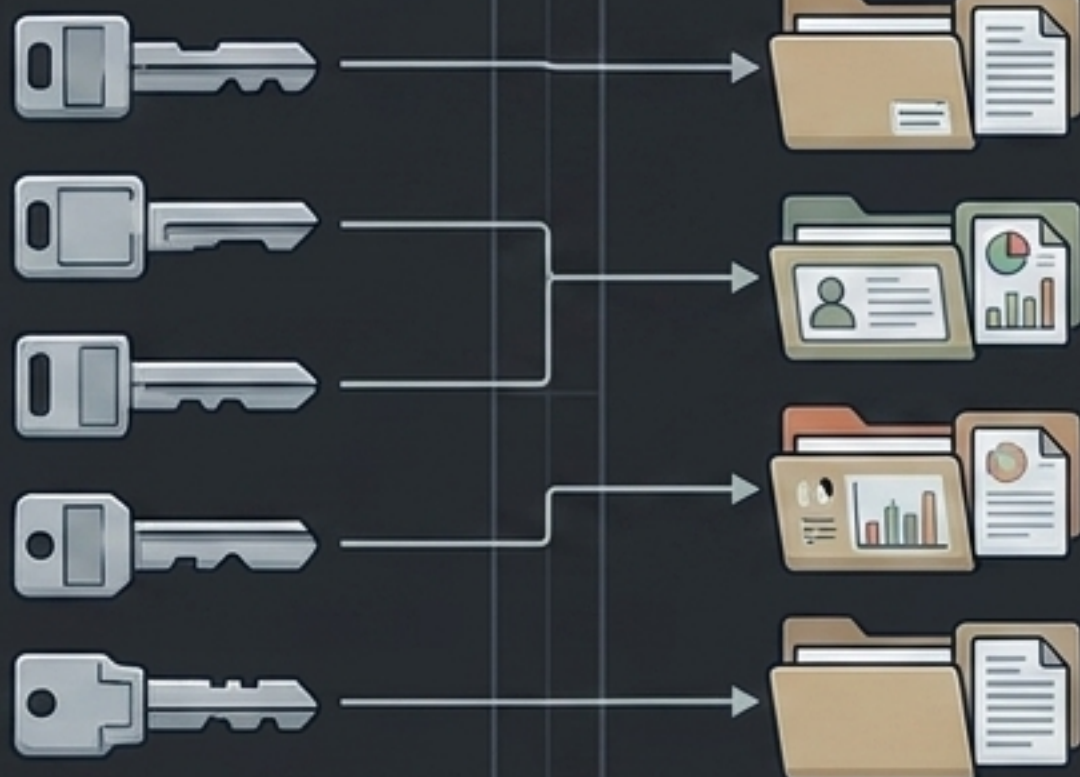
[Claves Únicas: SÍ]

[Valores Duplicados: SÍ]

[Orden: No (en HashMap)]

Clave (Key) - ej. DNI

Valor (Value) - ej. Datos Personales

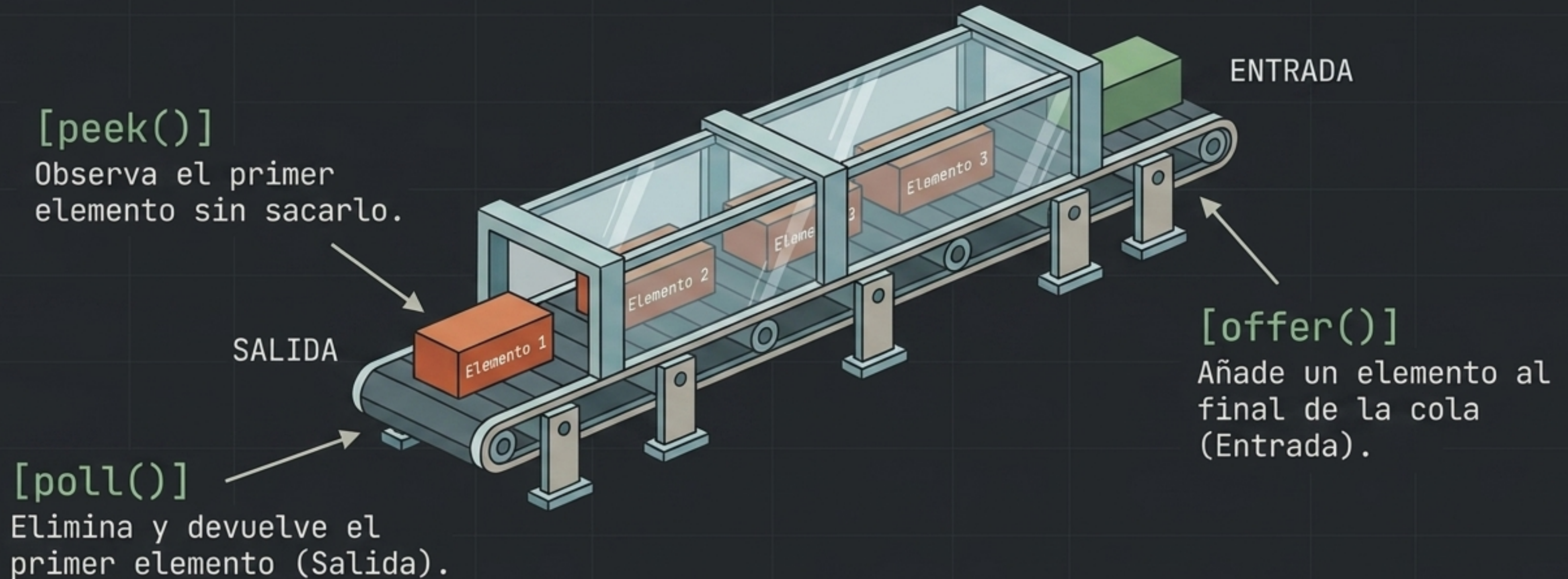


```
mapa.put("Jose Antonio", 30);
```

<code>put(K, V)</code>	: Asigna valor a clave
<code>get(K)</code>	: Recupera valor
<code>containsKey(K)</code>	: Verifica existencia
<code>remove(K)</code>	: Elimina entrada

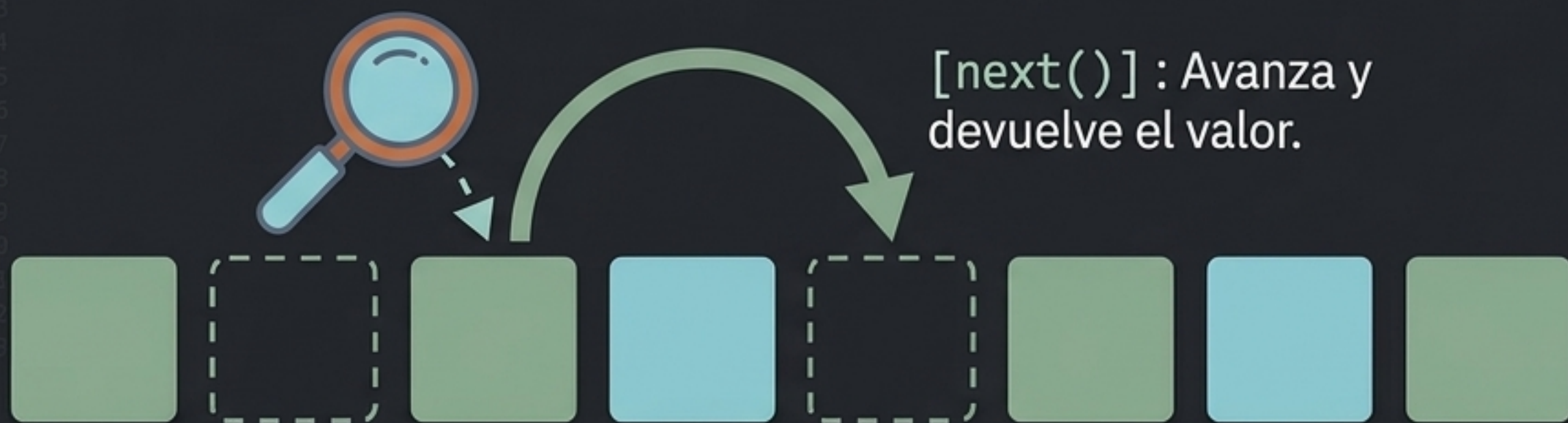
Interfaz Queue: Tuberías y Procesamiento FIFO

FIFO (First-In, First-Out). El primero en llegar es el primero en ser procesado.



Iteradores: Navegación Segura

Permiten recorrer y eliminar elementos de cualquier colección sin romper su estructura interna ni desbordar límites.



`[hasNext()] :`

Devuelve boolean si hay elementos.

`[next()] :`

Recupera el elemento.

`[remove()] :`

Elimina el elemento actual de forma segura.

El bucle **for-each** (`for(Tipo var : Colección)`) usa iteradores de fondo de forma transparente.

¿Qué Colección Utilizar? El Árbol de Decisión

